

Data Attribute Types

1. Nominal Attributes

اسمي

- **Definition:** Categories or labels with no order.
- **Example:** Hair color = {black, brown, blond, red}.
- **Note:** You cannot calculate mean or median, only mode (most frequent).

2. Binary Attributes

- **Definition:** Attributes with only two states (0/1 or True/False).
- **Example:** Smoker = {0 = does not smoke, 1 = smokes}.
- **Types:**
 - **Symmetric:** Both states equally important (e.g., Gender = male/female).
 - **Asymmetric:** One state more important (e.g., HIV test: 1 = positive, 0 = negative).

3. Ordinal Attributes

- **Definition:** Values have order but differences are not measurable.
- **Example:** Drink size = {small, medium, large}.
- **Note:** We know large > medium > small, but not how much bigger.

4. Numeric Attributes

كمي

- **Definition:** Quantitative, measurable values (integer or real).
- **Types:**
 - **Interval-scaled:** Meaningful differences, but no true zero.
 - **Example:** Temperature in Celsius (20°C is 5° higher than 15°C).
 - **Ratio-scaled:** Have a true zero and allow ratios.
 - **Example:** Weight (60 kg is twice 30 kg).

5. Discrete vs. Continuous Attributes

- **Discrete:** Finite or countable values.
 - **Example:** Number of children = {0,1,2,3...}.
- **Continuous:** Infinite real values (measured, not counted).
 - **Example:** Height = 170.5 cm, 180.2 cm.

Types of Data Sets

Data sets can be categorized into three main groups: *Record Data*, *Graph-Based Data*, and *Ordered Data*.

1. Record Data

- Most common type in data mining.
- Each record = a collection of attributes (fields).

Examples/Variations:

- **Transaction Data:** Each row = a customer's purchase (e.g., bread, milk, soda).
- **Data Matrix:** Numeric attributes stored in rows (objects) and columns (features).
- **Sparse Data Matrix:** Special form, such as document-term matrix, where only non-zero values are stored.

Tid	Refund	Marital Status	Taxable Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

(a) Record data.

TID	ITEMS
1	Bread, Soda, Milk
2	Beer, Bread
3	Beer, Soda, Diapers, Milk
4	Beer, Bread, Diapers, Milk
5	Soda, Diapers, Milk

(b) Transaction data.

Projection of xLoad	Projection of yLoad	Distance	Load	Thickness
10.23	5.27	15.22	27	1.2
12.65	6.25	16.22	22	1.1
13.54	7.23	17.34	23	1.2
14.27	8.43	18.45	25	0.9

(c) Data matrix.

	team	coach	play	ball	score	game	win	lost	timeout	season
Document 1	3	0	5	0	2	6	0	2	0	2
Document 2	0	7	0	2	1	0	0	3	0	0
Document 3	0	1	0	0	1	2	2	0	3	0

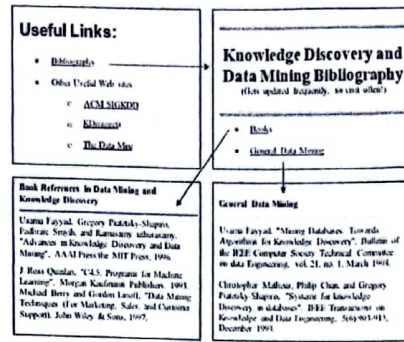
(d) Document-term matrix.

2. Graph-Based Data

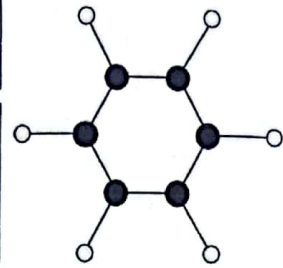
- Represents **relationships among objects** using nodes and edges.

Examples:

- Web pages** linked together by hyperlinks.
- Social networks** where people are nodes and interactions are links.
- Chemical structures** (e.g., benzene molecule) where atoms = nodes and bonds = edges.



(a) Linked web pages.



(b) Benzene molecule.

3. Ordered Data

مكاني أو الزمني الترتيب

- Data where attributes have **temporal or spatial order**.

Examples:

- Sequential Transaction Data:** Transactions with time (e.g., purchase history over days).
- Time Series Data:** Continuous measurements over time (e.g., monthly temperature records).
- Sequence Data:** Ordered items without time stamps (e.g., DNA sequence: A, T, G, C).
- Spatial & Spatio-Temporal Data:** Data linked to location (e.g., weather data collected across regions and time).

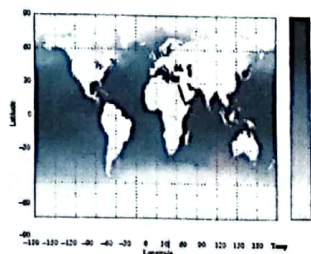
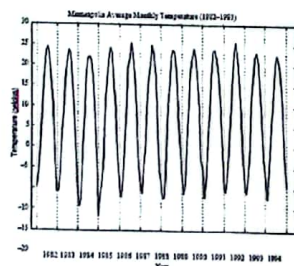
Time	Customer	Items Purchased
t1	C1	A, B
t2	C3	A, C
t2	C1	C, D
t3	C2	A, D
t4	C2	E
t5	C1	A, E

Customer	Time and Items Purchased
C1	(t1: A, B) (t2: C, D) (t5: A, E)
C2	(t3: A, D) (t4: E)
C3	(t2: A, C)

(a) Sequential transaction data.

GGTTCGCGCCTTCAGCCCCGCGCC
CGCAGGGCCCCGCCCCGCGCCGTC
GAGAAGGGCCCCGCTGGCGGGCG
GGGGGAGGCGGGGCCCGCCGAGC
CCAACCGAGTCCGACCAGGTGCC
CCCTCTGCTCGGCCTAGACCTGA
GCTCATTAGGCGGCAGCGGACAG
GCCAAGTAGAACACGCGAAGCGC
TGGGCTGCCTGCTGCGACCAGG

(b) Genomic sequence data.



4. Handling Non-Record Data

- Many algorithms are designed for record data.
- Solution:** Convert non-record data into record format by extracting features (e.g., chemical compound structures represented with binary features showing presence/absence of substructures).

Measures of Similarity and Dissimilarity

Similarity: is a **numerical measure** that expresses how much two objects are similar (متشابهان).

- **Range:** Usually between 0 and 1.
 - 0 → no similarity at all.
 - 1 → perfectly identical (maximum similarity).
 - Values in between (e.g., 0.6, 0.8) indicate partial similarity.

Example (Nominal Data): ◦ Object A: Hair Color = Black ◦ Object B: Hair Color = Black → Similarity = 1 (exact match). ◦ Object A: Hair Color = Black ◦ Object B: Hair Color = Blond → Similarity = 0 (completely different).	Example (Numeric Data): If similarity is calculated with a formula such as cosine similarity: Document A vector = (3, 2, 0) Document B vector = (6, 4, 0) → Similarity = 1 because the vectors point in the same direction
--	---

Dissimilarity: Dissimilarity measures how much two objects are **different**. It is often called **distance**.

- **Range:** From 0 to ∞ .
 - 0 → no difference (the objects are identical).
 - Higher values → greater difference.
- The closer the value is to 0, the more similar the objects are. Larger numbers mean the objects are farther apart in terms of attributes. لصفات حيث من أبعد

Example (Numeric Data): Consider two people's ages: ◦ Person A = 20 years ◦ Person B = 25 years → Dissimilarity = $25 - 20 = 5$. ◦ If both are 20 → Dissimilarity = 0 (identical ages).	Example (Binary Data): ◦ Patient A = Smoker (1) ◦ Patient B = Non-smoker (0) → Dissimilarity = 1 (different). If both are smokers (1,1) → Dissimilarity = 0.
---	--

Proximity: Proximity is a **general term** that refers to either similarity or dissimilarity.

- ✓ • If the analysis is about **closeness**, you can use similarity.
- ✓ • If the analysis is about **distance or separation**, you can use dissimilarity.
- Both capture the same relationship but from opposite perspectives.

- **Similarity** → higher is better (closer to 1 = more alike).
- **Dissimilarity** → lower is better (closer to 0 = more alike).

Dissimilarity of numeric data: Minkowski distance

The distance measures that are commonly used for computing the **dissimilarity** of objects described by **numeric attributes**. These measures include the Euclidean, Manhattan, and Minkowski distances.

1. Euclidean Distance

Definition: The straight-line distance between two points ("as the crow flies").

$$d(i, j) = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \dots + (x_{ip} - x_{jp})^2}$$

- **Example:**

For two points A(2,3) and B(5,7):

$$d(A, B) = \sqrt{(2 - 5)^2 + (3 - 7)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

2. Manhattan Distance (City Block Distance)

Definition: The sum of the absolute differences of the coordinates. Named because it represents moving along city blocks.

$$d(i, j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \dots + |x_{ip} - x_{jp}|$$

- **Example:**

For A(2,3) and B(5,7):

$$d(A, B) = |2 - 5| + |3 - 7| = 3 + 4 = 7$$

Both Euclidean and Manhattan distances satisfy four **important properties**:

- **Nonnegativity:** $d(i, j) \geq 0$. Distance is always non-negative.
- **Identity of Indiscernibles:** $d(i, i) = 0$. The distance of an object to itself is zero.
- **Symmetry:** $d(i, j) = d(j, i)$. Distance is the same in both directions.
- **Triangle Inequality:** $d(i, j) \leq d(i, k) + d(k, j)$. The direct distance between two objects is always less than or equal to going through a third object.

3. Supremum Distance (Chebyshev Distance, L_∞ Norm)

Definition: A special case of the Minkowski distance when $h \rightarrow \infty$

It measures the maximum absolute difference between the corresponding attributes of two objects.

- **Formula:**

$$d(i, j) = \max_f |x_{if} - x_{jf}|$$

- **Interpretation:** Instead of summing or squaring differences (like Euclidean or Manhattan), it only takes the **largest difference** among all attributes.

Example

Suppose we have two points:

- A = (2, 5, 7)
- B = (6, 3, 10)

Step 1: Compute absolute differences for each attribute:

- $|2 - 6| = 4$
- $|5 - 3| = 2$
- $|7 - 10| = 3$

Step 2: Take the maximum value:

$$d(A, B) = \max(4, 2, 3) = 4$$

4. Cosine Similarity

- **Definition:** Cosine similarity measures the cosine of the angle between two vectors.
 - If angle = $0^\circ \rightarrow$ similarity = 1 (vectors point in the same direction).
 - If angle = $90^\circ \rightarrow$ similarity = 0 (vectors are orthogonal, no similarity). عامودي
 - If angle = $180^\circ \rightarrow$ similarity = -1 (vectors point in opposite directions).
- **Formula:**

$$\text{sim}(x, y) = \frac{x \cdot y}{\|x\| \|y\|}$$

where $\|x\|$ is the Euclidean norm of vector $x = (x_1, x_2, \dots, x_p)$, defined as $\sqrt{x_1^2 + x_2^2 + \dots + x_p^2}$.

where $\cdot$ indicates vector dot product, $\|x\|$: the length of vector x

- **Example:** How Compute similarity between x and y ?

$x = (5, 0, 3, 0, 2, 0, 0, 2, 0, 0)$

$y = (3, 0, 2, 0, 1, 1, 0, 1, 0, 1)$

$$x \cdot y = 5 \times 3 + 0 \times 0 + 3 \times 2 + 0 \times 0 + 2 \times 1 + 0 \times 1 + 0 \times 0 + 2 \times 1 + 0 \times 0 + 0 \times 1 = 25$$

$$\|x\| = \sqrt{5^2 + 0^2 + 3^2 + 0^2 + 2^2 + 0^2 + 0^2 + 2^2 + 0^2 + 0^2} = 6.48$$

$$\|y\| = \sqrt{3^2 + 0^2 + 2^2 + 0^2 + 1^2 + 1^2 + 0^2 + 1^2 + 0^2 + 1^2} = 4.12$$

$$\text{sim}(x, y) = 0.94.$$

Simple program to compute Euclidean (L2) distance between two vectors.

```
import math
A = [2, 3, 8]
B = [5, 7, 9]
if len(A) != len(B):
    print("Error: vectors must have the same length.")
else:
    total = 0
    # نحسب الفروقات عنصر عنصر
    for i in range(len(A)):
        diff = A[i] - B[i]
        total += total + diff ** 2
    # الفرق بين العنصرين
    # نضيف مربع الفرق إلى المجموع
    dist = math.sqrt(total)
    # الجذر التربيعي للمجموع
    print("Euclidean distance =", dist)
```

Simple program to compute Manhattan distance between two vectors.

```
A = [2, 3, 4, 5]
B = [5, 7, 6, 9]

if len(A) != len(B):
    print("Error: vectors must have the same length.")
else:
    total = 0
    for i in range(len(A)):
        diff = abs(A[i] - B[i])
        total += diff
    # الفرق المطلق
    # نضيف الفرق إلى المجموع
    print("Manhattan distance =", total)
```

```
# Simple program to compute Supremum Distance (Chebyshev Distance)
A = [2, 3, 4, 5]
B = [5, 7, 6, 9]

if len(A) != len(B):
    print("Error: vectors must have the same length.")
else:
    max_diff = 0
    print("Step by step calculation:")
    for i in range(len(A)):
        diff = abs(A[i] - B[i]) # الفرق المطلق
        print(f"|{A[i]} - {B[i]}| = {diff}")
        if diff > max_diff: # نحتفظ بأكبر فرق
            max_diff = diff

    print("Chebyshev distance =", max_diff)
```

```
# Simple program to compute Cosine similarity between two vectors.
import math

X = [5, 0, 3, 0, 2]
Y = [3, 0, 2, 0, 1]

if len(X) != len(Y):
    print("Error: vectors must have the same length.")
else:
    # Step 1: dot product
    dot = 0
    for i in range(len(X)):
        dot += X[i] * Y[i]
        print(f"{X[i]} * {Y[i]} = {X[i]*Y[i]}")

    # Step 2: compute lengths
    len_x = math.sqrt(sum(xi**2 for xi in X))
    len_y = math.sqrt(sum(yi**2 for yi in Y))

    # Step 3: cosine similarity
    cosine_sim = dot / (len_x * len_y)

    print("Dot product =", dot)
    print("||X|| =", len_x)
    print("||Y|| =", len_y)
    print("Cosine similarity =", cosine_sim)
```

Using scikit-learn: provides ready-to-use functions for common distance and similarity measures. It is simple, efficient, and widely used in machine learning for tasks like clustering, classification, and text analysis.

```
from sklearn.metrics.pairwise import euclidean_distances, manhattan_distances, cosine_similarity
import numpy as np

A = np.array([[2, 3, 4]])
B = np.array([[5, 7, 6]])

print("Euclidean:", euclidean_distances(A, B))
print("Manhattan:", manhattan_distances(A, B))
print("Cosine similarity:", cosine_similarity(A, B))
```

K-Nearest Neighbors (k-NN) Classifier

- k-NN is one of the **simplest machine learning algorithms**.
- It is a **lazy learner**: it does not build a model, it just stores the training data.
- To predict a new data point:
 - Find the **k closest training samples** (neighbors).
 - Use their labels to make the prediction (majority vote in classification, average in regression).

2. How it Works

1. Choose a value for **k** (number of neighbors).
2. Measure the **distance** between the new point and all training points (usually Euclidean distance).
3. Select the **k nearest points**.
4. **Classification**: take the majority label among the neighbors.
Regression: take the average of their values.

3. Example (Concept)

This example demonstrates how the k-Nearest Neighbors (k-NN) algorithm works step by step using numerical data. We have a set of training points with labels and we want to classify a new point.

Training Data

Point (x, y)	Label (Class)
(1, 2)	Red
(2, 3)	Red
(3, 3)	Blue
(6, 5)	Blue
(7, 7)	Blue

New point to classify: (3, 2)

Step 1: Calculate Distances

We calculate Euclidean distance from the new point (3, 2) to all training points:

$$d((3, 2), (1, 2)) = \sqrt{(3-1)^2 + (2-2)^2} = \sqrt{4} = 2$$

$$d((3, 2), (2, 3)) = \sqrt{(3-2)^2 + (2-3)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41$$

$$d((3, 2), (3, 3)) = \sqrt{(3-3)^2 + (2-3)^2} = \sqrt{1} = 1$$

$$d((3, 2), (6, 5)) = \sqrt{(3-6)^2 + (2-5)^2} = \sqrt{9+9} = \sqrt{18} \approx 4.24$$

$$d((3, 2), (7, 7)) = \sqrt{(3-7)^2 + (2-7)^2} = \sqrt{16+25} = \sqrt{41} \approx 6.40$$

Step 2: Sort Neighbors by Distance

1. (3, 3) → Blue → distance = 1
2. (2, 3) → Red → distance ≈ 1.41
3. (1, 2) → Red → distance = 2
4. (6, 5) → Blue → distance ≈ 4.24
5. (7, 7) → Blue → distance ≈ 6.40

Step 3: Apply k-NN Rule

If $k = 1$ → Closest neighbor = (3,3) → Blue

If $k = 3$ → Closest 3 neighbors = {Blue, Red, Red} → Majority = Red

```
from sklearn.neighbors import KNeighborsClassifier
```

```
# Training data (features + labels)
```

```
X = [[1,2], [2,3], [3,3], [6,5], [7,7]]
```

```
y = ["Red", "Red", "Blue", "Blue", "Blue"]
```

```
# إنشاء k-NN مع k=3
```

```
knn = KNeighborsClassifier(n_neighbors=3)
```

```
knn.fit(X, y)
```

```
# نقطة جديدة للتصنيف
```

```
new_point = [[3,2]]
```

```
prediction = knn.predict(new_point)
```

```
print("Prediction for", new_point, "=", prediction[0])
```